

// Структура данных между вершинным и пиксельным шейдерами

struct VS_OUTPUT

{

float4 Pos : SV_POSITION;

float2 Tex : TEXCOORD0;

};

// Вершинный шейдер

VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)

{

VS_OUTPUT output = (VS_OUTPUT)0;

output.Pos = Pos;

output.Tex = Tex;

return output;

}

// === ДВА НЕЗАВИСИМЫХ ТЕКСТУРНЫХ СЛОТА ===

Texture2D srcTexture : register(t0); // Слот t0: Исходная картинка или предыдущий буфер

Texture2D statsTexture : register(t1); // Слот t1: Скрытые данные аналитики Stage3

SamplerState samplerState0 : register(s0);

// Буфер констант параметров кадра

cbuffer cbData : register(b0)

{

float width;

float height;

float d_width;

float d_height;

float splitX;

float padding;

};

```

// Константа весов яркости из 2011 года
static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

// =====
// 3-Й ШЕЙДЕР: Статистический анализ блоков 4x4 (Pass 1)
// =====

float4 PS(VS_OUTPUT input) : SV_Target
{
    float GrayMid = 0.0f;
    float GrayMin = 1.0f;
    float GrayMax = 0.0f;
    float GraySqr = 0.0f;
    float2 tex_coord;

    [unroll]
    for (int i_width = -1; i_width < 3; i_width++)
    {
        for (int i_height = -1; i_height < 3; i_height++)
        {
            tex_coord.x = input.Tex.x + float(i_width) * d_width;
            tex_coord.y = input.Tex.y + float(i_height) * d_height;

            float4 k1 = srcTexture.Sample(samplerState0, tex_coord);
            float gray = dot(k1.rgb, WEIGHT);

            GrayMid += gray;
            if (gray < GrayMin) GrayMin = gray;
            if (gray > GrayMax) GrayMax = gray;
        }
    }
}

```

```
GrayMid /= 16.0f;
```

```
[unroll]
```

```
for (int j_width = -1; j_width < 3; j_width++)
```

```
{
```

```
    for (int j_height = -1; j_height < 3; j_height++)
```

```
    {
```

```
        tex_coord.x = input.Tex.x + float(j_width) * d_width;
```

```
        tex_coord.y = input.Tex.y + float(j_height) * d_height;
```

```
        float4 k1 = srcTexture.Sample(samplerState0, tex_coord);
```

```
        float gray = dot(k1.rgb, WEIGHT);
```

```
        GraySqr += (gray - GrayMid) * (gray - GrayMid);
```

```
    }
```

```
}
```

```
GraySqr = sqrt(GraySqr / 16.0f);
```

```
return float4(GrayMid, GrayMin, GrayMax, GraySqr);
```

```
}
```

```
// =====
```

```
// 4-Й ШЕЙДЕР: Укрупненный анализ макроблоков 8x8 (Pass 2)
```

```
// =====
```

```
float4 PS_Stage2(VS_OUTPUT input) : SV_Target
```

```
{
```

```
    float Summ_mid = 0.0f;
```

```
    float Summ_sqr = 0.0f;
```

```
    float Summ_min = 1.0f;
```

```
    float Summ_max = 0.0f;
```

```
    float2 tex_coord;
```

```

[unroll]
for (int i_width = 0; i_width < 2; i_width++)
{
    for (int i_height = 0; i_height < 2; i_height++)
    {
        tex_coord.x = input.Tex.x + float(i_width) * (d_width * 4.0f);
        tex_coord.y = input.Tex.y + float(i_height) * (d_height * 4.0f);

        float4 k1 = srcTexture.Sample(samplerState0, tex_coord);

        Summ_mid += k1.x;
        if (k1.y < Summ_min) Summ_min = k1.y;
        if (k1.z > Summ_max) Summ_max = k1.z;
        Summ_sqr += k1.x * k1.x + k1.w * k1.w;
    }
}
Summ_mid /= 4.0f;
Summ_sqr = sqrt(abs(Summ_sqr / 4.0f - Summ_mid * Summ_mid));

return float4(Summ_mid, Summ_min, Summ_max, Summ_sqr);
}

// =====
// 5-й ШЕЙДЕР: Безопасный однопроходный макроанализ блоков 3x3 (Pass 3)
// =====

float4 PS_Stage3(VS_OUTPUT input) : SV_Target
{
    // 1. Читаем ИСХОДНЫЙ полноразмерный гладкий пиксель кадра
    float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
    float src_gray = dot(src_color.rgb, WEIGHT);

```

```

// 2. БЕЗОПАСНОЕ ВЫЧИСЛЕНИЕ МАКРОСТАТИСТИКИ БЛОКА 3x3

float macro_mid = 0.0f;
float macro_min = 1.0f;
float macro_max = 0.0f;
float macro_sqr = 0.0f;
float2 tex_coord;

// Сканируем компактное окно 3x3 пикселя вокруг текущей точки
[unroll]
for (int i_width = -1; i_width <= 1; i_width++)
{
    for (int i_height = -1; i_height <= 1; i_height++)
    {
        // Рассчитываем координаты и принудительно зажимаем их в границы [0.0, 1.0] для защиты
        // от краша!
        tex_coord.x = saturate(input.Tex.x + float(i_width) * d_width);
        tex_coord.y = saturate(input.Tex.y + float(i_height) * d_height);

        float4 k1 = srcTexture.Sample(samplerState0, tex_coord);
        float gray = dot(k1.rgb, WEIGHT);

        macro_mid += gray;
        if (gray < macro_min) macro_min = gray;
        if (gray > macro_max) macro_max = gray;
    }
}
macro_mid /= 9.0f; // Усредняем по блоку 3x3 (9 пикселей)

// Расчет локальной дисперсии макроблока 3x3
[unroll]
for (int j_width = -1; j_width <= 1; j_width++)
{
    for (int j_height = -1; j_height <= 1; j_height++)

```

```

{
    tex_coord.x = saturate(input.Tex.x + float(j_width) * d_width);
    tex_coord.y = saturate(input.Tex.y + float(j_height) * d_height);

    float4 k1 = srcTexture.Sample(samplerState0, tex_coord);
    float gray = dot(k1.rgb, WEIGHT);
    macro_sqr += (gray - macro_mid) * (gray - macro_mid);
}
}

macro_sqr = sqrt(macro_sqr / 9.0f);

// 3. ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ЛОКАЛЬНОГО КОНТРАСТА С ЗАЩИТОЙ ИЗ ШАГА 131
[INDEX_124, INDEX_131]

float delta = src_gray - macro_mid;
float local_dispersion = abs(macro_mid - global_mid);

// Защитный знаменатель барьера (12.75f перевели в масштаб [0.0 - 1.0])
float denom = local_dispersion + (12.75f / 255.0f);
float local_contrast = delta / denom;

// ЖЕСТКОЕ ОГРАНИЧЕНИЕ ДИАПАЗОНА СРЕЗА (ТЕПЕРЬ И В ШЕЙДЕРЕ!)
if (local_contrast < -1.5f) local_contrast = -1.5f;
if (local_contrast > 1.5f) local_contrast = 1.5f;

// Финальный синтез дефектоскопа с честным множителем 1.8f
float out_gray = macro_mid + local_contrast * (local_dispersion * 1.8f);

// Возвращаем чистый, сбалансированный монохромный результат
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// =====
// 6-Й ШЕЙДЕР: Чистый вывод шторки "До / После" (Pass 4)

```

```

// =====

float4 PS_Final(VS_OUTPUT input) : SV_Target
{
    // 1. Читаем ИСХОДНЫЙ полноразмерный монохромный кадр (слот t0)
    float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
    float src_gray = dot(src_color.rgb, WEIGHT);

    // ИНТЕЛЛЕКТУАЛЬНАЯ ШТОРКА СРАВНЕНИЯ:
    // =====
    // >>> МЫ ИСПРАВЛЯЕМ МАТЕМАТИКУ РАЗДЕЛЕНИЯ НА БАЗЕ НАШЕЙ ПЕРЕМЕННОЙ splitX:
    // =====
    // Если пиксель находится в ЛЕВОЙ половине экрана — выводим чистый оригинал
    if (input.Tex.x < splitX)
    {
        return float4(src_gray, src_gray, src_gray, 1.0f);
    }

    // --- ПРАВАЯ ПОЛОВИНА: Выводим уже готовый, отфильтрованный гладкий кадр из Stage3 ---
    // Читаем результат работы 5-го шейдера из специального слота t1
    float4 filtered_color = statsTexture.Sample(samplerState0, input.Tex);
    float gray_filtered = filtered_color.x;

    // Применяем финальную гамма-коррекцию из 2011 года для проявления деталей
    float corrected_gray = pow(abs(gray_filtered), 1.0f / 2.2f);
    corrected_gray = corrected_gray * 1.05f - 0.02f;
    corrected_gray = saturate(corrected_gray);

    // Рисуем тонкую разделительную черную линию ровно по положению шторки
    if (abs(input.Tex.x - splitX) < 0.002f)
    {
        return float4(0.0f, 0.0f, 0.0f, 1.0f);
    }
}

```

```

}

// Выводим гладкий результат высокой четкости
return float4(corrected_gray, corrected_gray, corrected_gray, 1.0f);
}

// =====
// ДОПОЛНИТЕЛЬНЫЙ ШЕЙДЕР: Качественное сжатие кадра ровно в 2 раза (Downsample)
// =====
Texture2D inputTexture : register(t0); // Входная текстура предыдущего масштаба

float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target
{
    // d_width и d_height хранят шаг одного пикселя (1.0 / размер_входной_текстуры)
    float2 offset = float2(d_width, d_height);

    // Считываем блок 2x2 соседних пикселей
    float4 p00 = inputTexture.Sample(samplerState0, input.Tex);
    float4 p10 = inputTexture.Sample(samplerState0, input.Tex + float2(offset.x, 0.0f));
    float4 p01 = inputTexture.Sample(samplerState0, input.Tex + float2(0.0f, offset.y));
    float4 p11 = inputTexture.Sample(samplerState0, input.Tex + offset);

    // Возвращаем строгое среднее арифметическое без замыливания
    return (p00 + p10 + p01 + p11) / 4.0f;
}

// =====
// НОВЫЙ ЧИСТЫЙ ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ КАДРА
// =====
float4 PS_Copy(VS_OUTPUT input) : SV_Target
{
    // Просто считываем пиксель один в один без фильтрации и изменений

```

```
    return srcTexture.Sample(samplerState0, input.Tex);  
}
```